International Journal of Scientific Research in Science and Technology
Available online at : www.ijsrst.com
OPEN ACCESS
Print ISSN: 2395-6011 | Online ISSN: 2395-602X
doi : https://doi.org/10.32628/IJSRST25125127

Advancing Agentic AI through Communication Protocols

Aniket P. Kakde, Karan M. Bhoyar, Muhammad Aiman Shad, Prof. Sudesh A. Bachwani
Department of Computer Science & Engineering, JDIET, Yavatmal, Maharashtra, India

ARTICLEINFO

Article History:

Accepted : 01 Sept 2025
Published: 05 Oct 2025

Publication Issue :
Volume 12, Issue 5
September-October-2025

Page Number :
299-308

ABSTRACT

Autonomous agents powered by Large Language Models (LLMs) require reliable and standardized frameworks to connect tools, exchange contextual information, and synchronize tasks across diverse systems. Despite growing interest in such agents, current integration with external tools remains disjointed. Developers often have to manually create interfaces, handle authentication protocols, and navigate incompatible function-calling standards across platforms. To overcome these limitations and promote the evolution of agentic AI, it is critical to establish standardized communication protocols that ensure interoperability—enabling agents and systems to seamlessly discover each other's capabilities, share data, and coordinate operations.

This paper explores a structured overview of emerging communication standards for agents, focusing on the Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP). MCP utilizes a JSON-RPC based client-server architecture to enable secure execution of tools and well-typed data transfer. ACP introduces a REST-compliant message structure with support for asynchronous streaming and multipart formats, facilitating rich, multimodal agent outputs.A2A enables agents to delegate tasks peer-to-peer using capability-rich Agent Cards, enabling scalable and distributed workflows across organizations. ANP facilitates agent discovery and secure collaboration in open networks, leveraging decentralized identifiers (DIDs) and semantic graphs based on JSON-LD.

Keywords: Large Language Models (LLMs), Agent Communication, Interoperability Protocols, Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), Agent Network Protocol (ANP), Autonomous Agents, Multimodal Messaging, Decentralized Identity (DID), Agentic AI

Copyright © 2025 The Author(s): This is an open-access article distributed under the terms of the Creative Commons Attribution **4.0 International License (CC BY-NC 4.0)**

I. INTRODUCTION

A. The Rise of Agentic AI

Modern artificial intelligence is undergoing a profound paradigm shift, moving beyond supervised tools to embrace autonomous systems capable of pursuing complex, long-term goals with minimal human intervention. This new frontier, termed Agentic AI, represents a qualitative leap from conventional AI, which depends on structured instructions and close oversight. Agentic systems are defined by a set of core characteristics that enable their autonomy: the ability to formulate novel strategies (autonomy); deconstruct and pursue multifaceted objectives (goal complexity); operate and adapt in dynamic, real-world environments (environmental complexity); make context-aware decisions (independent decision-making); and maintain effectiveness when faced with novel circumstances (adaptability).

This evolution is built upon several key technical foundations. Reinforcement Learning (RL) provides a framework for agents to develop sophisticated, goal-directed strategies through environmental interaction. Goal-oriented architectures offer a modular structure for planning and executing complex, multi-step tasks. However, the most catalytic advance has been the enhancement of Large Language Models (LLMs) with the ability to reason, plan, and interact with the external world. Techniques such as Retrieval-Augmented Generation (RAG) grounded model outputs in external knowledge, while the introduction of "Function Calling" by OpenAI in 2023 was a watershed moment. It provided a structured method for LLMs to invoke external APIs, transforming them from passive text generators into active agents capable of orchestrating tools and services.

B. The Communication Bottleneck: A Critical Barrier to Collective Intelligence

Despite these advances, the full potential of Agentic AI is constrained by a significant "communication bottleneck". The current ecosystem is a fragmented patchwork of ad-hoc integrations and proprietary standards, creating critical barriers to progress. Key challenges include the lack of a universal mechanism for agents to discover each other's capabilities, the creation of communication silos by heterogeneous development frameworks like LangChain and LlamaIndex, and the absence of context standardization, which forces developers to write brittle, bespoke code for every tool integration.

This fragmentation prevents the formation of a connected network of intelligence, analogous to the state of computing before the universal adoption of TCP/IP and HTTP. Without standardization, agents remain isolated, unable to collaborate effectively to solve complex, real-world problems. The academic and industrial consensus is that formal, standardized protocols are not merely a technical convenience but a fundamental necessity to unlock the next stage of AI evolution: collective intelligence. The rapid, parallel emergence of multiple protocol proposals in late 2024 and 2025 from major industry players like Anthropic, Google, and IBM is a direct market response to this critical deficiency in the AI infrastructure stack, signaling an urgent need to solve the interoperability problem to sustain innovation

C. A Historical Perspective on Agent Interoperability

To appreciate the design of modern protocols, it is essential to understand their historical predecessors. Early efforts in the 1990s focused on symbolic agent languages for knowledge-based systems. The Knowledge Query and Manipulation Language (KQML) introduced speech-act primitives and a flexible message envelope, which was later refined by the Foundation for Intelligent Physical Agents' Agent Communication Language (FIPA-ACL), which standardized performative semantics and interaction protocols. However, their complexity and reliance on heavyweight XML-style encodings limited their adoption primarily to academic and defense applications. The 2000s saw the rise of Service-Oriented Architectures (SOA) and web services

(SOAP/WSDL), which enabled service decoupling but often resulted in high operational complexity and brittle, custom adapters. These foundational systems were not designed for the dynamic, semantic, and scalable needs of modern LLM-powered agents. Table I provides a timeline of key milestones, illustrating the trajectory from these early efforts to the protocol-oriented solutions of today.

TABLE I

Year	Milestone	Key Contribution
1993	KQML	Introduced speech-act primitives and a flexible message envelope for knowledge-based agents.
1998	MASIF	Defined basic service registration and discovery mechanisms for agent environments.
2000	FIPA-ACL	Standardized performative semantics, content languages, and interaction protocols with formal pre-/post-conditions
2002	Web Services (SOAP/WSDL)	Enabled service-oriented agent integration via UDDI, XML messaging, and contract definitions.
2020	RAG	Coupled dense vector retrieval with LLM decoding to ground outputs in external corpora.
2023	Function Calling	Allowed LLMs to emit JSON-formatted API calls against a catalog of function schemas.
2024	MCP	Proposed a JSON-RPC protocol for standardized context ingestion and tool invocation.
2024	ANP	Peer-to-peer protocol enabling cross-platform and cross-organization agent communication over the open internet.
2024	ACP	Defined performative messaging primitives with formal types and security layers.
2025	A2A	Introduced peer discovery, capability exchange, and decentralized agent dialogues.

II. METHODS AND MATERIAL : A REVIEW OF CORE PROTOCOLS

This section provides a technical review of the four primary protocols designed to solve the agent interoperability problem. Each protocol's architecture and methodology are detailed, representing the core "materials" of this comparative study. The architectural choices of these protocols reflect distinct philosophies on trust and control, ranging from centralized, managed systems suitable for enterprise

environments to fully decentralized models designed for the open web.

A. Model Context Protocol (MCP): Standardizing Agent-to-Tool Communication

The Model Context Protocol (MCP), an open standard initiated by Anthropic in November 2024, standardizes how a single AI agent interacts with external tools, data sources, and resources. Often described as the "USB-C for AI," it provides a universal interface that eliminates the need for bespoke integration code for each new tool.

MCP's architecture follows a client-server model governed by the JSON-RPC 2.0 protocol, enabling structured, language-agnostic communication. As shown in Fig. 1, the architecture consists of three components: the

MCP Host (the AI application, such as a chatbot or IDE), the **MCP Client** (a component within the host that connects to a server), and the **MCP Server** (a program that exposes capabilities to the agent). This model decouples the LLM's reasoning from direct tool execution, enhancing security by preventing sensitive credentials like API keys from being exposed to the model itself. Communication occurs over a flexible transport layer that supports high-performance local interaction via stdio and remote interaction via HTTP with Server-Sent Events (SSE) for streaming. The server exposes three core capabilities:

Tools for invoking external APIs, Resources for accessing structured and unstructured data, and Prompts for using predefined templates and workflows.

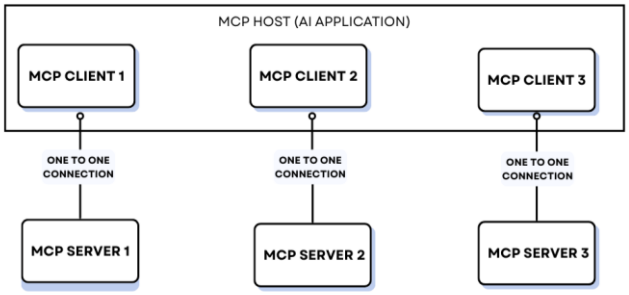


Fig 1 MCP High Level Architecture

B. Agent-to-Agent (A2A) Protocol: Enabling Enterprise Collaboration

The Agent-to-Agent (A2A) Protocol is an open standard, initiated by Google and now governed by the Linux Foundation, designed for complex, stateful collaboration between AI agents, typically within trusted organizational boundaries. It provides a common language for agents built with different frameworks to delegate tasks and coordinate workflows.

A2A is built on a client-server model using JSON-RPC 2.0 over HTTP(S) and supports synchronous, streaming (SSE), and asynchronous interactions, making it well-suited for long-running tasks. The protocol's key concepts, illustrated in Fig. 2, revolve around three primary objects. The **Agent Card** is a JSON metadata document that describes an agent's identity, capabilities, and endpoint, enabling discovery. A

Task is a stateful unit of work with a unique ID that progresses through a defined lifecycle (e.g., submitted, working, completed). An **Artifact** is a tangible output generated by an agent during task processing, such as a document or structured data. This structure facilitates robust, auditable delegation between specialized agents in an enterprise setting.

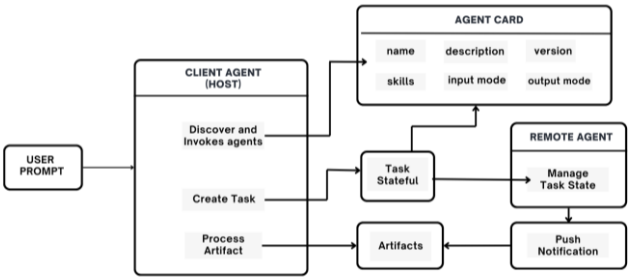


Fig 2 A2A Protocol Architecture

C. Agent Communication Protocol (ACP): A REST-Native Framework for Interoperability

The Agent Communication Protocol (ACP) is a REST-native open standard from IBM Research, also under the Linux Foundation, that focuses on standardizing asynchronous-first communication in local or

enterprise systems. Its design philosophy emphasizes loose coupling and a clear separation of concerns.

As depicted in Fig. 3, ACP uses a three-role, brokered architecture. The **Agent Client** initiates requests. The **ACP Server** acts as a protocol broker, maintaining an Agent Registry for discovery and enforcing system-wide policies like authentication. The **ACP Agent** is the execution endpoint containing the domain-specific logic. Discovery is flexible, supporting both a centralized registry and decentralized manifest files hosted at a /.well-known/agent.yml endpoint. A key feature of ACP is its structured, multi-part message format, where each part is explicitly typed using MIME types, allowing for rich, multimodal data exchange (e.g., text, images, video) within a single request.

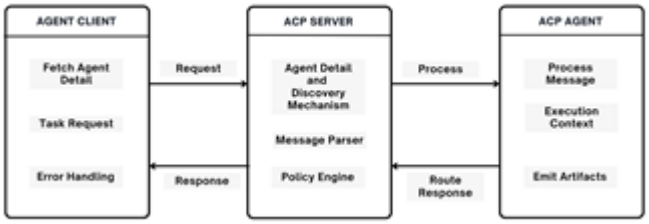


Fig. 3 An Overview of ACP

D. Agent Network Protocol (ANP): A Vision for a Decentralized "Internet of Agents"

The Agent Network Protocol (ANP) is the most ambitious of the four, proposing a decentralized, peer-to-peer (P2P) communication standard designed for cross-platform agent interoperability over the open internet. Its vision is to create an "Internet of Agents" where autonomous systems can discover, authenticate, and interact without relying on a central authority.

ANP is built on a layered P2P architecture, as shown in Fig. 4, that incorporates principles from decentralized and semantic web technologies. The **Identity and Encrypted Communication Layer** uses W3C Decentralized Identifiers (DIDs) to establish trustless, cryptographic identity and end-to-end encrypted communication. The **Meta-Protocol Layer** introduces a novel mechanism for agents to autonomously negotiate and coordinate communication protocols using natural language. The **Application Protocol Layer** uses JSON-LD for semantic capability descriptions via an Agent Description Protocol (ADP), allowing agents to programmatically understand each other's functions. This decentralized approach aims to foster a truly open and scalable agent ecosystem.

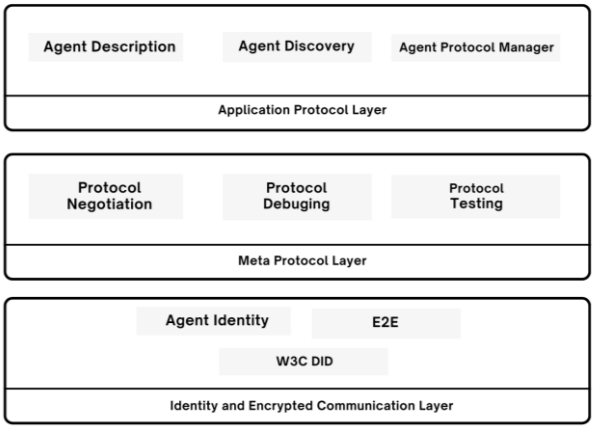


Fig. 3.6 An Overview of ANP

III.RESULTS AND DISCUSSION

This section transitions from technical description to a synthetic analysis of the protocols. It begins with a practical use case to demonstrate their synergy, followed by a direct comparative analysis, an examination of recent ecosystem developments, and a critical review of the current security landscape

A. Protocol Synergy in a Multi-Agent Workflow: A Use Case Analysis

The four protocols are not mutually exclusive competitors but rather complementary components of a layered communication stack. This synergy is best illustrated through a coherent use case: an AI-powered travel assistant tasked with planning a complex trip. The user prompt is: "Plan a 5-day trip for me from Mumbai to Paris next month. Find the best flights, book a 4-star hotel near the Eiffel Tower, and suggest some popular tour packages, including some with brochures."

The workflow, visualized in Fig. 5, unfolds across several stages, with each protocol serving a distinct purpose:

1. **Tool Interaction (MCP):** The primary Travel Assistant Agent acts as an MCP Host to perform simple, stateless data retrieval. It connects to a "Flight Search Server" and a "Weather API Server" to gather initial data on flight options and the expected weather in Paris.
2. **Inter-Agent Delegation (A2A):** Booking a hotel is a complex, stateful task. The Travel Assistant acts as an A2A Client, discovers a specialized "Hotel Booking Agent" within its enterprise via its Agent Card, and delegates the entire booking process by sending a formal Task. The Hotel Booking Agent handles the multi-step interaction and returns a confirmation Artifact.
3. **Rich Content Retrieval (ACP):** To find internal tour packages with rich media like brochures, the agent uses ACP, which excels at multimodal communication. It queries the enterprise's ACP Server (Agent Registry), discovers an "Internal Tours Agent," and receives a multi-part message containing text descriptions, images, and video links.
4. **Cross-Domain Discovery (ANP):** To find external tour options, the agent leverages ANP to search the open internet. It discovers a "Paris Adventures Agent" from a different company, verifies its identity using its DID, retrieves its capabilities via its ADP document, and interacts to get additional tour packages.

Finally, the orchestrating agent aggregates the information from all sources—flights from MCP, hotel confirmation from A2A, internal tours from ACP, and external tours from ANP—to synthesize a complete itinerary for the user. This example demonstrates a clear functional stratification: MCP for local tool use, A2A and ACP for trusted enterprise collaboration, and ANP for open-world discovery.

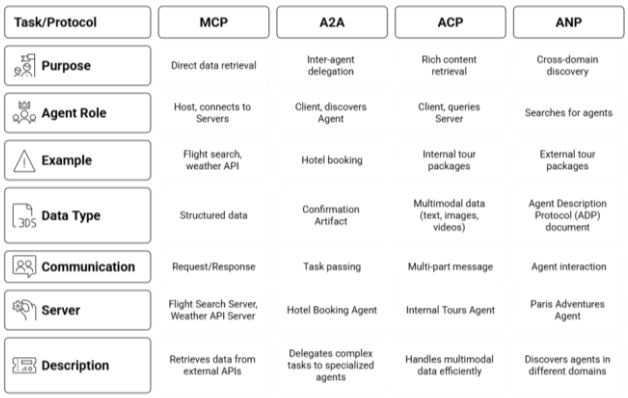


Fig. 5 Protocol Interaction

B. Comparative Analysis: Architectures, Scopes, and Trade-offs

A direct comparison of the protocols, summarized in Table II, highlights their distinct design choices and intended domains. The architectural spectrum, from the centralized client-server model of MCP to the fully decentralized P2P model of ANP, directly reflects the intended operational environment and trust model. MCP and ACP are designed for controlled settings where a central authority (the host application or registry server) manages discovery and security. A2A operates in a semi-decentralized manner within a trusted enterprise network. ANP, in contrast, is built for the untrusted, open web, where cryptographic identity via DIDs is necessary to establish trust between unknown parties.

TABLE II COMPARATIVE ANALYSIS OF AGENT PROTOCOLS

Aspects	MCP (Model Context Protocol)	ACP (Agent Communication Protocol)	A2A (Agent-to-Agent Protocol)	ANP (Agent Network Protocol)
Architecture	Client Server	Brokered Client-	Peer-like Client-	Decentralized

Aspects	MCP (Model Context Protocol)	ACP (Agent Communication Protocol)	A2A (Agent-to-Agent Protocol)	ANP (Agent Network Protocol)
	(JSON-RPC)	Server (REST)	Server	Peer-to-Peer
Primary Use Case	Single agent interacting with external tools and data sources	Heterogenous agents communicating within a local or enterprise system	Multi-agent workflows and task delegation within trusted organizational boundaries	Cross-platform agent discovery and collaboration over the open internet
Discovery	Manual registration or static URL lookup	Centralized Agent Registry	Agent Card retrieval via HTTP at a well-known endpoint	Decentralized discovery via search agents and directories
Identify and Auth	Token-based; no built-in identity standard	Bearer tokens,mutual, TS,JWS	Out-of-band headers;designed for trusted enterprise contexts	Decentralized Identifiers (DIDs) for trustless identity
Message Format	JSON-RPC 2.0 with primitives for Tools,Resources,Prompts	Structured multipart messages with MIME typed parts	JSON-RPC 2.0 with Task,Message,and Artifact objects	JSON-LD with semantic schemas (Schema.org)
Strengths	Tight LLM integration;standardized context injection	Multimodal messaging;vendor-neutral;strong observability	Rich inter-agent negotiation;artifact-driven delegation	Trustless identity;AI-native protocol negotiation;open by design
Limitations	Centralized server assumption; potential for prompt injection	Requires a central registry; assumes server control	Enterprise-centric;assumes a trusted agent catalog	High negotiation overhead; ecosystem is still evolving

C. Ecosystem Evolution and Industry Adoption (2025)

The agent protocol landscape is not static; it is evolving and consolidating at a remarkable pace. By early 2025, MCP saw rapid and widespread adoption, with major AI providers including OpenAI and Google DeepMind, as well as cloud platforms like Microsoft Azure, integrating support for the protocol. This swift uptake has established MCP as the de facto industry standard for agent-to-tool communication, the foundational layer for connecting a single agent to its resources.

In the inter-agent communication layer, a landmark development occurred in September 2025 when IBM's

ACP team announced it was joining forces with Google's A2A protocol team to develop a unified standard. This move signals a strong industry push towards consolidation, aiming to prevent fragmentation and create a single, robust protocol for structured collaboration within trusted enterprise environments. Furthermore, these protocols are already serving as a foundation for higher-level applications. The announcement of the Agent Payments Protocol (AP2) as an extension of A2A demonstrates how standardized communication is enabling new economic activities, such as secure, auditable, agent-led commerce.

D. Critical Challenges and the Security Landscape

The rapid pace of protocol development and adoption has significantly outstripped the maturation of corresponding security practices, creating a dangerous gap between capability and safety. While each protocol has inherent theoretical risks—such as MCP's susceptibility to prompt injection, A2A's reliance on a trusted network, and ANP's negotiation overhead—a series of real-world security incidents in mid-2025 has proven these risks to be tangible and urgent.

Security researchers discovered thousands of misconfigured MCP servers exposed on the open web, creating a massive attack surface. High-profile vulnerabilities were exploited in production systems, including prompt injection attacks against Atlassian's AI agent and a flaw in GitHub's official MCP server that had far-reaching consequences. Flawed MCP implementations led to significant data leaks at companies like Asana, where one organization's data was exposed to another. Perhaps most alarmingly, the "EchoLeak" vulnerability was discovered in Microsoft 365 Copilot, a zero-click exploit that leveraged design flaws in RAG-based systems to exfiltrate data from a user's context. These events underscore that security is the primary lagging factor in the agentic ecosystem. The drive for functionality has left a trail of vulnerabilities, making trust and security the single greatest challenge to widespread and safe adoption.

IV. CONCLUSION AND FUTURE DIRECTIONS

A. Conclusion

The transition from isolated AI tools to a collaborative ecosystem of autonomous agents is a pivotal undertaking, gated by the need for standardized communication. The ad-hoc integrations of the past are giving way to a more structured and capable future, built upon a new generation of open protocols. This analysis has shown that the four primary protocols—MCP, A2A, ACP, and ANP—are not competing for a single role but are instead self-organizing into a complementary, layered architecture. This emerging functional stack sees MCP as the foundational layer for agent-to-tool interaction, the consolidated A2A/ACP standard as the robust framework for intra-enterprise collaboration, and ANP as the ambitious vision for a decentralized, open-web "Internet of Agents."

The path toward true agent interoperability is now clearer than ever. However, the journey is fraught with peril. The significant and demonstrated security vulnerabilities that have emerged represent the most critical threat to this vision. Without a concerted effort to build robust security, governance, and trust frameworks, the full potential of collective AI intelligence may be undermined by a loss of enterprise and consumer confidence. The future of Agentic AI depends not only on how well agents can talk to each other, but on how safely they can do so.

B. Future Directions

The evolution of agent protocols is expected to continue, moving from solving immediate integration problems to building the infrastructure for a global network of collective intelligence. Key future directions include:

1) **Evolvable and Adaptive Protocols:** Future systems may move beyond static specifications, enabling agents to dynamically negotiate or learn optimal protocol variations to enhance communication efficiency for specific tasks.

2) **Privacy-Preserving Communication:** As agents handle increasingly sensitive data, there will be a critical need for protocols that incorporate privacy-preserving techniques, such as those inspired by federated learning, to allow collaboration on insights without exposing raw data.
3) **The Agent Data Network (ADN):** A long-term vision is the development of a dedicated data infrastructure optimized for agent-native communication. Unlike the human-centric web, an ADN would support structured, intent-driven, and protocol-compliant information exchange, enabling persistent state and long-horizon planning for a global agent network.
4) **Security and Trust as a Research Frontier:** Given the current landscape, the most urgent area for future work is security. This includes developing universal, cross-protocol standards for identity and authorization, creating tools for automated security auditing of agent capabilities, and designing robust sandboxing environments to contain and mitigate the risks of tool and agent execution.

REFERENCES

[1]. D. B. Acharya, K. Kuppan, and B. Divya, "Agentic AI: Autonomous Intelligence for Complex Goals-A Comprehensive Survey," IEEE Access, vol. 13, pp. 18912-18936, 2025.

[2]. S. J. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 3rd ed. Prentice Hall, 2010.

[3]. Y. Yang et al., "A Survey of AI Agent Protocols," arXiv preprint arXiv:2504.16736, 2025.

[4]. A. Ehtesham, A. Singh, S. Kumar, and G. K. Gupta, "A Survey of Agent Interoperability Protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)," arXiv preprint arXiv:2505.02279, 2025.

[5]. L. Wang et al., "A survey on large language model based autonomous agents," Frontiers of Computer Science, vol. 18, no. 6, Mar. 2024.

[6]. X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions," arXiv preprint arXiv:2503.23278, 2025.

[7]. OpenAI, "Function calling," OpenAI Platform, 2023. [Online]. Available: https://platform.openai.com/docs/guides/function-calling.

[8]. Google, "Agent2Agent (A2A) Protocol," 2025. [Online]. Available: https://a2a-protocol.org/latest/.

[9]. IBM BeeAI, "Introduction to Agent Communication Protocol (ACP)," 2024. [Online]. Available: https://docs.beeai.dev/acp/alpha/introduction.

[10]. Agent Network Protocol Contributors, "Agent Network Protocol (ANP)," 2024. [Online]. Available: https://github.com/agent-network-protocol/AgentNetworkProtocol.

[11]. P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive nlp tasks," in Advances in Neural Information Processing Systems, vol. 33, pp. 9459-9474, 2020.

[12]. S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP), 2023.

[13]. H. Chase, "LangChain," 2022. [Online]. Available: https://github.com/langchain-ai/langchain.

[14]. M. R. Genesereth and S. P. Ketchpel, "The KQML protocol: A specification of language and communication," in Proc. Third Int. Conf. on Information and Knowledge Management (CIKM), 1993.

[15]. Foundation for Intelligent Physical Agents, "FIPA Communicative Act Library Specification," 2000. [Online]. Available: https://www.fipa.org/specs/fipa00037/SC00037J.html.